TEAM 04

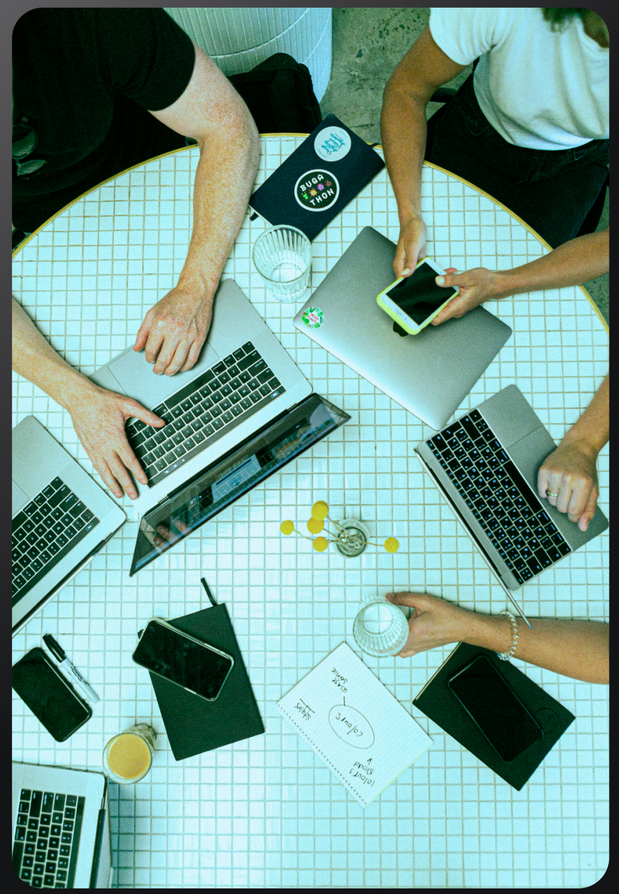
Retrospective X

sea|me
software engineering in automotive
and mobility ecosystems

DrivaPi

Sprint Goal

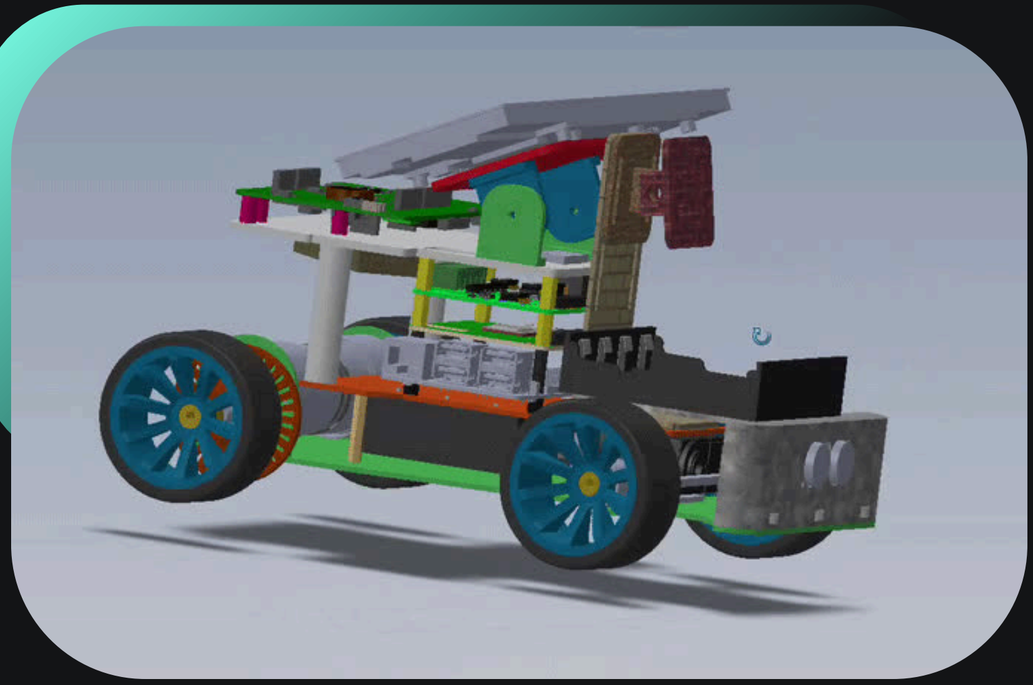
Successfully develop an **MVP for robust lane detection** by training **ML algorithm**, integrate **smooth PID motor control** and **power monitoring** using the INA231 module, while maintaining strict **real-time performance** through Qt **optimization** and TraceX telemetry.



DrivaPi

CAR REDESIGN

Stepping into the autonomous driving module, we wanted the car to be **as close as possible** to its **final form**, while remaining lightweight and maneuverable. To achieve this, we **completely redesigned the whole structure** to fit alll the components more **efficiently**, trading all the metal parts for lighter 3D printed ones, and creating a 1:1 exact model of the car.
Unfortunately, TUMO was too busy to print our parts so we can’t build the car.



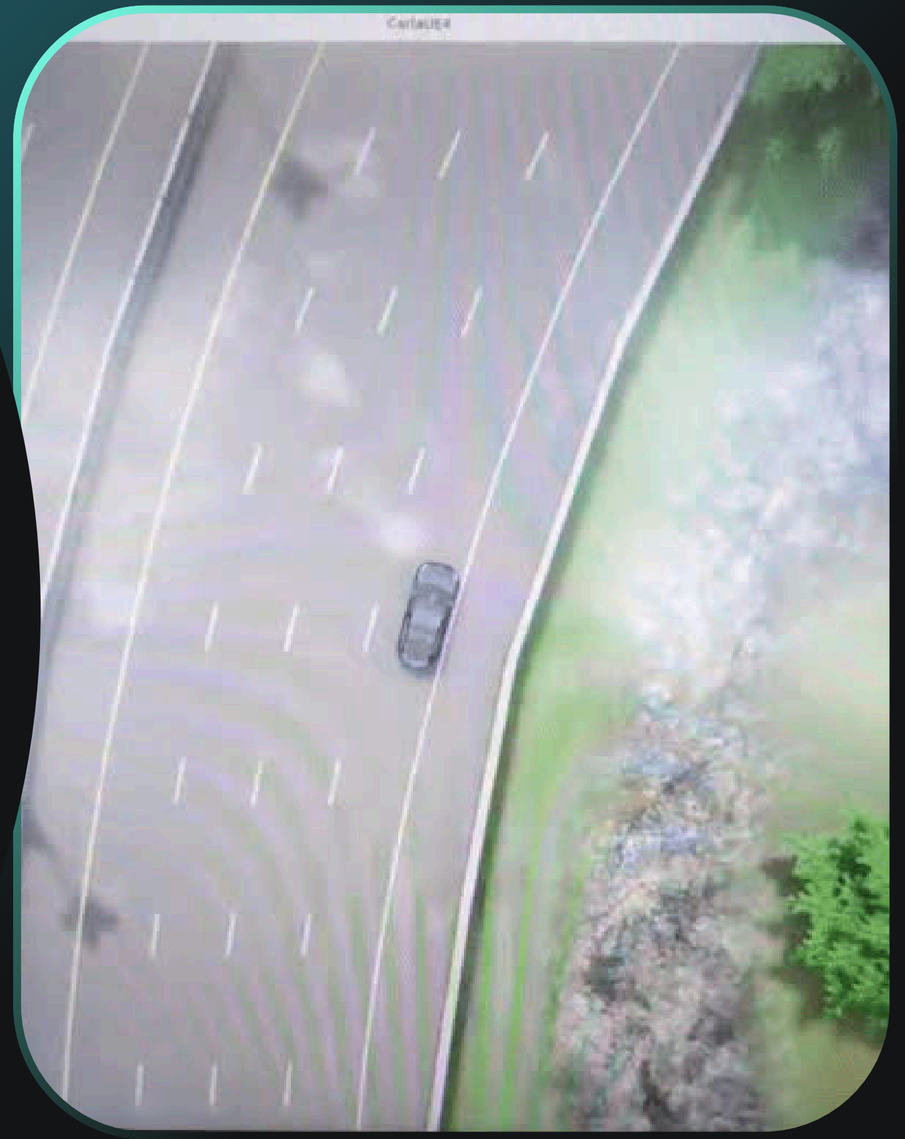
ADAS
Lane Detection

After **countless hours of training** and testing **different models** (YOLO, UFLD, and LaneATT), we **successfully detected multiple different lanes** under various settings, including at night and with adverse conditions in CARLA, but also at SEA:ME Labs track.

DrivaPi

Autonomous Driving & RTPP

By utilizing aided **Imitation Learning** as the initial training for our **decision-making model,** we successfully developed a very basic **MVP** that **drives autonomously** on CARLA, executing simple actions like steering, accelerating or decelerating. Further training is **necessary** to ensure full, safe autonomy.

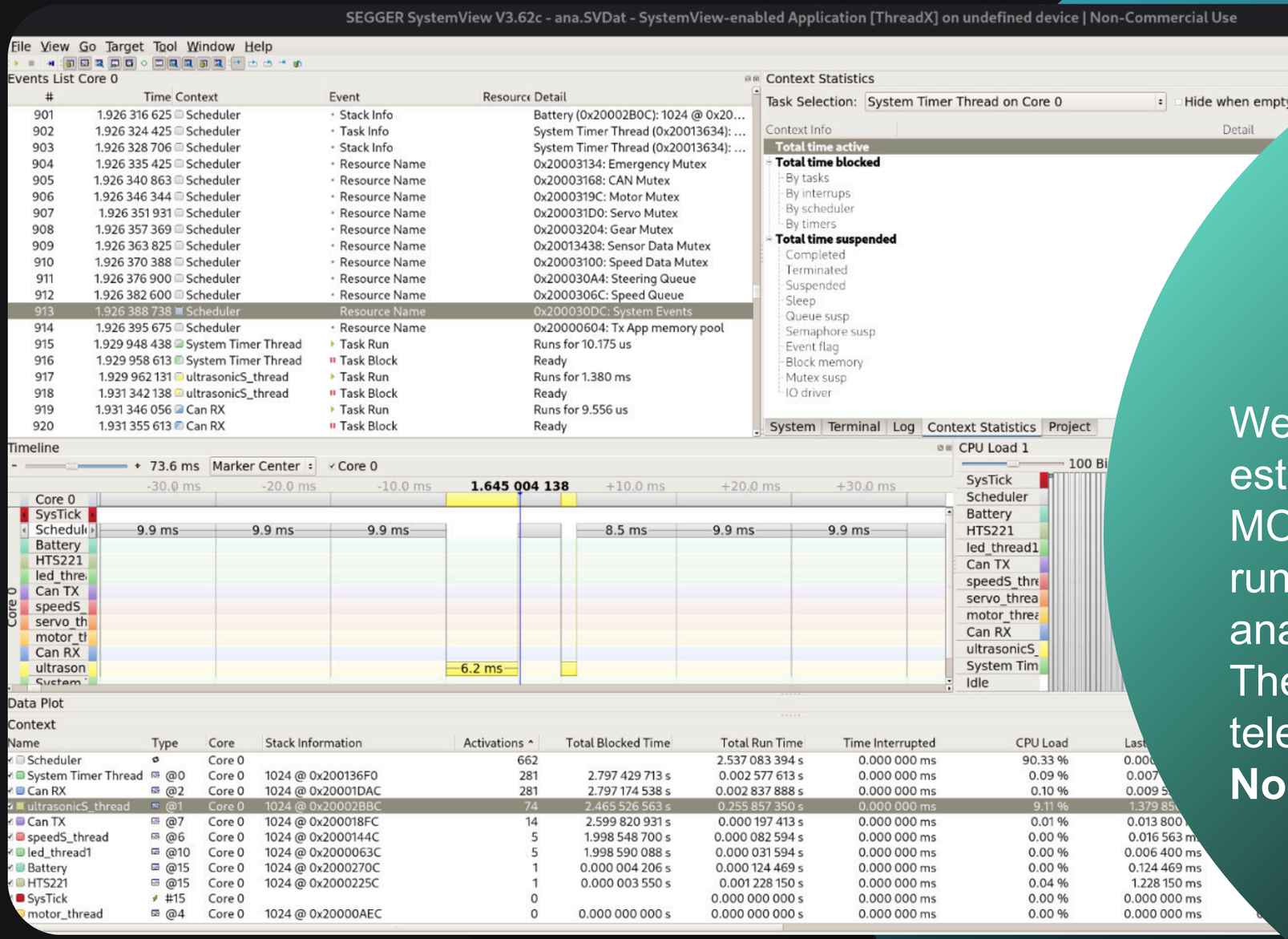


Motor Control

PID

We are implementing a **PID-based speed** control in STM32 firmware. As opposed to the previous system, the MCU now receives an **order to match the speed** (rather than pre-set PWM values) requested by the Raspberry Pi via CAN, executing it **seamlessly** and **smoothly**. It still needs some tuning and testing.

DrivaPi



SEGGER

We used **Open On-Chip Debugger** to establish a connection with STM32 MCU to create a script that captures runtime information to then be later analyzed in **SEGGER SystemView**.
The final goal is to gain thread telemetry information like **CPU Load**, **No. of Activations**, **TRT**, **TTI**, etc.

Dashboard

Qt

Qt’s code safety was **significantly improved**. Defensive programming was enforced on the CAN-to-KUKSA feeder before allowing it to touch the PiRacer's software stack. **Multiple failsafes** were introduced to **strengthen error parsing**, **preventing data poisoning** and **enhancing lifecycle safety**. We **minimized CPU usage** by migrating scripts to C++, and **boosted network performance** thorugh bi-dir. gRPC Stream.

DrivaPi

Technical Problems & Solutions

INA231

Our INA231 power monitor burned while we were testing the battery connections and soldering everything in place.
We will need a new transistor or module.

TraceX

Implementing TraceX turned out to be a a very hard challenge because it's heavily reliant on Microsoft's systems.
We are implementing SEGGER instead.

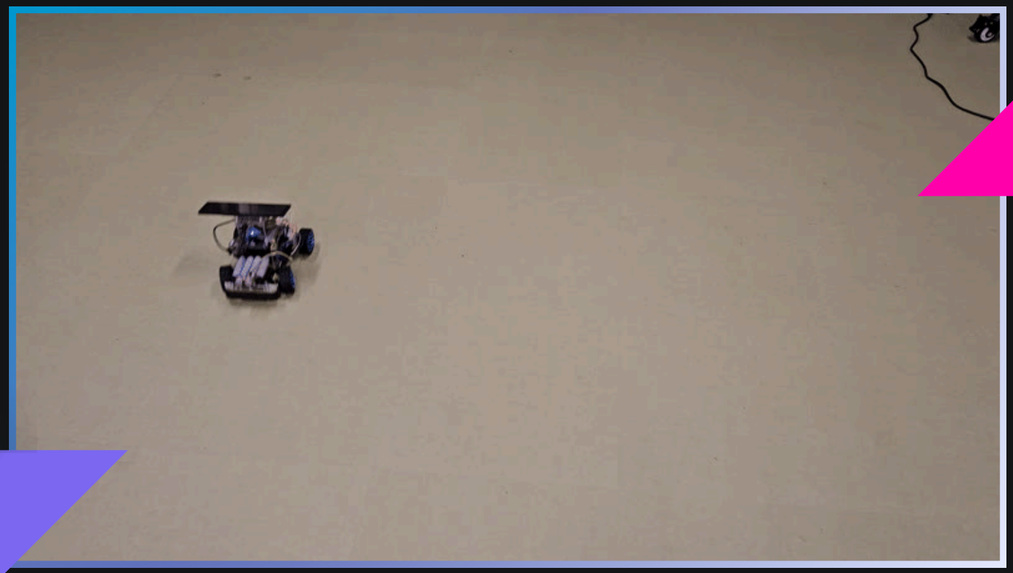
New Battery

We applied a quick fix to our existing (broken) battery pack, which allowed it to keep working.
No need to buy a new one, but a BMS will be implemented.

Qt Dashboard

Accepting tens of thousands of lines in multiple PRs with merge conflicts took longer than expected.
Regardless, lots of optimization was done on-PR.

THANK YOU!



DrivaPi